**National University of Computer & Emerging Sciences, Karachi**
**Spring 2026**
**Assignment- 2**
**27 April, 2026**

| | |
|---|---|
| **Course Code: CS2009** | **Course Name: Compiler Construction (CS-4031)** |
| **Instructor Name:** | |
| **Date of Submission** | **06 May, 2026** |

**Instructions:** **Max Marks: 100**

- Make your Assignments on Full Scale Papers, Typed Assignments will not be acceptable:
- 20% penalty for 1 day late
- 40% penalty for 2 days late
- Submission not allowed afterwards

## Q1. Attribute Classification — S, L, Both, or Neither

For each part below, examine the grammar production and the associated semantic rule. Identify whether the attribute on the left-hand side (or as indicated) is:

| S — Synthesized | L — Inherited | Both S and L | Neither |
|---|---|---|---|

Part (a)

| (a) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (a) | E → E1 + T | E.val = E1.val + T.val | Is E.val a Synthesized (S) or Inherited (L) attribute? |

Part (b)

| (b) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (b) | T → T1 * F | T.type = T1.type | Is T.type a Synthesized (S) or Inherited (L) attribute? |

Part (c)

| (c) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (c) | D → T id | id.type = T.type | Is id.type a Synthesized (S) or Inherited (L) attribute? |

Part (d)

| (d) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (d) | A → B C | B.env = A.env | Is B.env a Synthesized (S) or Inherited (L) attribute? |

Part (e)

| (e) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (e) | E → - E1 | E.val = 0 - E1.val | Is E.val S, L, both, or neither? |

Part (f)

| (f) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (f) | S → id := E | S.ok = (id.type == E.type) | Is S.ok a Synthesized (S) or Inherited (L) attribute? |

Part (g)

| (g) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (g) | L → L1, id | id.type = L.type; L.len = L1.len + 1 | Identify the type of each attribute: L.type, id.type, L.len |

Part (h)

| (h) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (h) | P → D S | D.table = emptyTable(); S.table = D.table | Is S.table a Synthesized (S) or Inherited (L) attribute? |

Part (i)

| (i) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (i) | F → (E) | F.val = E.val | Is F.val a Synthesized (S) or Inherited (L) attribute? |

Part (j)

| (j) | Grammar Production | Semantic Rule | Answer |
|---|---|---|---|
| (j) | C → if E then S1 else S2 | S1.in = C.in; S2.in = C.in; C.out = E.true ? S1.out : S2.out | Classify: C.in, S1.in, C.out — are they S, L, both, or neither? |

## Q2. Syntax-Directed Definitions (SDD) — Annotated Parse Tree & Dependency Graph

Consider the following SDD for arithmetic expression evaluation:

| Production Rule | Semantic Rule |
|---|---|
| E → E1 + T | E.val = E1.val + T.val |
| E → E1 - T | E.val = E1.val - T.val |
| E → T | E.val = T.val |
| T → T1 * F | T.val = T1.val * F.val |
| T → F | T.val = F.val |
| F → ( E ) | F.val = E.val |
| F → digit | F.val = digit.lexval |

Given the expression: 3 + 4 * 5 - 2

Part (a) — Construct the Annotated Parse Tree

Draw the full parse tree for the expression 3 + 4 * 5 - 2 based on the grammar above. Annotate every node with its .val attribute, computed bottom-up using the semantic rules.

Part (b) — Construct the Dependency Graph

Draw the dependency graph for the annotated parse tree constructed above. Each node in the graph should represent an attribute instance. Draw directed edges from each attribute instance that is used to compute another.

## Q3. Abstract Syntax Tree (AST) Construction

Consider the following grammar for assignment and conditional statements:

```
S → id := E | if E then S | if E then S else S | while E do S
E → E + E | E * E | E > E | ( E ) | id | num
```

For each of the following statements, construct the Abstract Syntax Tree (AST). The AST should retain only semantically meaningful nodes, eliminating unnecessary grammar symbols (parentheses, keywords, etc.).

Part (a)

x := (a + b) * c

Construct the AST for the above assignment statement.

Part (b)

if x > 0 then y := x * 2 else y := 0

Construct the AST for the above if-else statement.

Part (c)

while a > b do a := a - b

Construct the AST for the above while loop statement.

## Q4. Print-Statement Semantic Actions — Parse Tree Construction

Consider the following grammar for a simple print statement:

| Production | Semantic Action / Rule |
|---|---|
| S → print ( L ) | { print(L.items) } |
| L → L1 , E | { L.items = L1.items ++ [E.val] } |
| L → E | { L.items = [E.val] } |
| E → E1 + T | { E.val = E1.val + T.val } |
| E → T | { E.val = T.val } |
| T → num | { T.val = num.lexval } |
| T → id | { T.val = lookup(id.name) } |

Given the statement: print ( x + 3 , 5 )

Part (a) — Construct the Parse Tree with Semantic Actions

Draw the complete parse tree for the above print statement. Annotate the nodes with the semantic actions executed at each production.

Part (b) — Trace the Evaluation

Assuming x = 7, trace the bottom-up evaluation of the parse tree and determine what is printed. Show all intermediate .val computations and L.items list construction step by step.

## Q5. Syntax-Directed Translation (SDT) — Translation Schemes

A Syntax-Directed Translation scheme (SDT) embeds program fragments (semantic actions) directly into grammar productions. Unlike SDD, the position of the action within a production matters.

Part (a) — Infix to Postfix Translation

Consider the following SDT for translating infix arithmetic expressions to postfix notation:

```
E → E1 + T   { print('+') }
E → E1 - T   { print('-') }
E → T
T → T1 * F   { print('*') }
T → F
F → digit    { print(digit.lexval) }
```

(i) Apply the SDT to the expression: 5 - 3 + 2 * 4

Show the complete parse tree and trace the order in which actions are executed. Write the final postfix output.

Part (b) — Type-Checking SDT

Write an SDT scheme (grammar + embedded semantic actions) to perform basic type checking for the following declaration and assignment grammar:

```
P → D ; S
D → type id
S → id := E
E → id | num | E + E
```

Your SDT must: (1) record the declared type of id in a symbol table, (2) determine the type of expression E, (3) emit an error if the type of E does not match the declared type of id.

Part (c) — SDT vs SDD Comparison

Answer the following short questions about the relationship between SDT and SDD:

(i) What is the key difference between an SDD and an SDT?

(ii) Under what conditions can an SDT with actions only at the right-end of productions be easily converted to a bottom-up parser?

(iii) Can every SDD be implemented as an SDT? Justify briefly.